

Direcionamento dos Projetos Finais – Turma iOS

1. Regras gerais do Projeto Final

- **Quantidade de equipes:** 5 equipes
- **Formação das equipes:** até 6 integrantes.
- **Tempo de apresentação:** 10 minutos de apresentação + até 5 minutos para perguntas, totalizando até 15 minutos.
- **Entrega:** disponibilizar o projeto no GitHub com README explicando o projeto, funcionalidades, tecnologias utilizadas, como executar e integrantes da equipe.
- **Escopo esperado:** um MVP funcional, com dados simulados quando necessário.

2. Requisitos obrigatórios para todos os projetos

Obrigatório	Direcionamento
Interface e navegação	Usar SwiftUI com NavigationStack, incluindo rotas, modais e fluxos condicionais entre telas.
Quantidade de telas	Mínimo de 5 telas e máximo de 8 telas.
API	Consumir pelo menos uma API REST usando URLSession e Codable, com tratamento de erros e estados de loading. Pode ser pública, mockada, própria ou gerenciada por Firebase/Supabase.
Persistência local	Gerenciar dados localmente usando CoreData, cache ou persistência de estado, considerando sincronização online/offline quando aplicável.
Arquitetura	Estruturar o projeto em MVVM no contexto do SwiftUI, separando View, ViewModel, repositórios, modelos e camada de dados, com injeção de dependência e fluxo de dados unidirecional.
README	Explicar proposta, funcionalidades, prints ou GIFs, arquitetura, dependências, instruções de execução e integrantes.

3. Estrutura mínima recomendada do aplicativo

- **Tela 1 - Login/Autenticação:** entrada do usuário; pode usar autenticação fake, JWT mockado ou serviço real.
- **Tela 2 - Home/Dashboard:** resumo principal, saldo, pontos, cartões, orçamento, carteira ou indicadores do tema escolhido.
- **Tela 3 - Listagem/Histórico/Extrato:** lista de registros com filtros, categorias ou ordenação.
- **Tela 4 - Cadastro/Ação principal:** criar despesa, simular investimento, reservar saque, solicitar cartão, cadastrar assinatura etc.

- **Tela 5 - Detalhe/Confirmação:** visualização detalhada ou confirmação de uma operação simulada.
- **Telas opcionais 6 a 8:** perfil, configurações, gráficos, favoritos, notificações, metas, participantes, segurança ou relatórios.

4. Projetos sugeridos

As equipes podem escolher um dos projetos abaixo para implementar.

1. Provisionamento de Saque

Proposta	Reserva de valor para saque em unidade específica.
Telas sugeridas	Login; saldo/menu; extrato; reserva de valor; escolha da unidade; confirmação.
API e persistência	API de unidades/caixas; persistência de reservas e histórico.
Diferencial	Fluxo bancário claro com confirmação e comprovante fake.

2. Programa Fidelidade

Proposta	Troca e envio de pontos entre clientes.
Telas sugeridas	Login; home com pontos e promoções; troca de pontos; envio de pontos; extrato; atualizar dados.
API e persistência	API de produtos/promoções; persistência de extrato e dados do cliente.
Diferencial	Carrossel de promoções e simulação de troca por produtos, serviços ou desconto na fatura.

3. Gerenciador de Cartões

Proposta	Gestão de cartões de crédito e operações simuladas.
Telas sugeridas	Login; lista de cartões; informações do cartão; solicitar cartão; bloquear/alterar limite; compras.
API e persistência	API de cartões/produtos; persistência de cartões favoritos e ações locais.
Diferencial	Cartão virtual com animação/visualização de CVV e ações rápidas.

4. SplitPay

Proposta	Divisão inteligente de contas entre pessoas.
Telas sugeridas	Grupos; despesas; participantes; cálculo de dívidas; pagamentos fake; histórico.
API e persistência	API de usuários/grupos mock; persistência de grupos e despesas.

Diferencial	Cálculo automático: quem deve quanto para quem.
-------------	---

5. SafeVault

Proposta	Cofre financeiro digital para dados sensíveis simulados.
Telas sugeridas	Login PIN/biometria; cartões; documentos; senhas; chaves Pix; perfil/segurança.
API e persistência	API de backup/mock; persistência local segura/simulada.
Diferencial	Bloqueio automático ao minimizar e proteção contra screenshots.

6. PocketBank Kids

Proposta	Educação financeira gamificada para crianças.
Telas sugeridas	Cofrinho; metas; missões; recompensas; avatar; ranking familiar; histórico.
API e persistência	API de missões/recompensas; persistência de progresso.
Diferencial	Evolução estilo jogo conforme a criança economiza.

7. Crypto Wallet Fake

Proposta	Carteira simulada de criptomoedas.
Telas sugeridas	Dashboard; ativos; cotação; conversão; compra/venda fake; watchlist; alertas.
API e persistência	API de cotações cripto; persistência de carteira e favoritos.
Diferencial	Oscilação em tempo real simulada ou via API.

8. PayFlow

Proposta	Organizador de assinaturas e gastos recorrentes.
Telas sugeridas	Assinaturas; vencimentos; notificações; categorias; histórico; resumo mensal; economia.
API e persistência	API de serviços/preços mock; persistência de assinaturas.
Diferencial	Mostra gastos com assinaturas pouco utilizadas.

5. Critérios de escopo e qualidade

- Priorizar fluxo completo e estável em vez de muitas telas incompletas.
- Tratar estados de carregamento, sucesso, erro e vazio nas telas principais.
- Usar dados persistidos localmente para que o app continue útil após fechar e abrir novamente.
- Usar uma API de forma visível no app com dados mockados.
- Garantir que cada tela tenha propósito claro e esteja conectada ao fluxo principal do MVP.

6. Checklist antes da apresentação

- ☐ Equipe possui 5 ou 6 integrantes identificados no README.
- ☐ App possui entre 5 e 8 telas implementadas.
- ☐ NavigationStack e fluxo entre telas estão funcionando sem travamentos no fluxo principal.
- ☐ Há consumo de pelo menos uma API.
- ☐ Há persistência local dos dados principais.
- ☐ Arquitetura MVVM está visível na estrutura de pacotes.
- ☐ README contém descrição, funcionalidades, integrantes, tecnologias e instruções de execução.
- ☐ Apresentação está ensaiada para caber em 10 minutos.

7. Roteiro sugerido para os 10 minutos de apresentação

Tempo	Conteúdo
1 min	Problema e proposta do app.
1 min	Equipe, papéis e tecnologias utilizadas.
4 min	Demonstração do fluxo principal do aplicativo.
2 min	Arquitetura MVVM, API consumida e persistência local.
1 min	Diferencial do projeto e decisões de escopo.
1 min	GitHub/README, aprendizados e encerramento.